LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – Inheritance



GEUGEUT NYARIKAWANTI SURAHMAT

103132400002

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

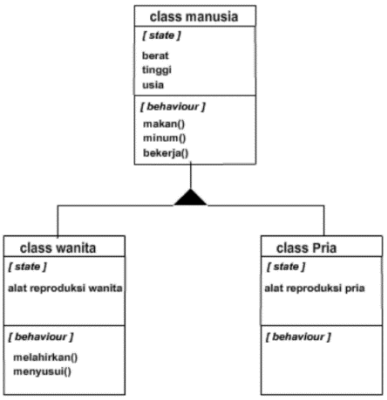
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Inheritance

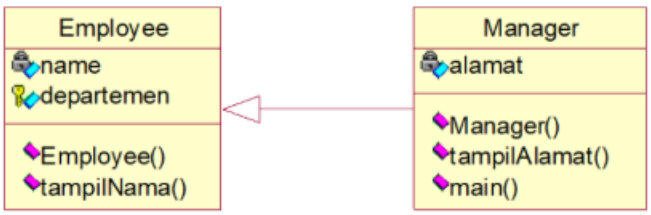
Inheritance merupakan suatu cara untuk menurunkan suatu kelas yang lebih umum menjadi suatu kelas yang lebih spesifik. Java mendukung inheritance dengan memungkinkan satu kelas untuk bersatu dengan kelas lainnya ke dalam deklarasinya. Notasi nya sebagai berikut:



Dalam inheritance terdapat 2 istilah yang sering digunakan, yaitu *superclass* dan *subclass*. *Superclass* adalah kelas yang menurunkan/memberikan karakteristik tertentu sedangkan *subclass* adalah kelas yang mendapatkan karakteristik tersebut. Pada contoh di atas, kelas manusia menjadi *superclass* sedangkan kelas wanita dan pria menjadi *subclass.* Karakteristik yang dimaksud adalah “berat”, “tinggi”, “usia”, “makan()”, “minum()”, dan “bekerja()”.

2. Keyword Super

Keyword super digunakan untuk merefer superclass dari suatu kelas, yaitu untuk merefer member dari suatu superclass, baik atribut maupun method. Berikut adalah visualisasi dari kode yang diimplementasikan pada guided bagian Keyword Super:



3. Polymorphism

Dalam Object Oriented Programming, kelas-kelas yang terbentuk dapat memiliki hubungan satu dengan yang lainnya, sesuai dengan kondisi dari kelas-kelas yang bersangkutan. Ada beberapa jenis hubungan yang dapat terjadi antara kelas yang satu dengan kelas yang lainnya, antara lain:

a. Overloading Method

Pembuatan dua atau lebih method dengan nama yang sama dalam satu kelas, dengan jumlah dan tipe data argumen masing – masing method berbeda.

b. Overriding Method

Di dalam java, jika dalam suatu subclass Anda mendefinisikan sebuah method yang sama dengan yang dimiliki oleh superclass, maka method yang Anda buat dalam subclass tersebut dikatakan meng-override superclass-nya. Sehingga jika Anda mencoba memanggil method tersebut dari instance subclass yang Anda buat, maka method milik subclass-lah yang akan dipanggil, bukan lagi method milik superclass-nya.

B. Guided

1. Inheritance

a. A.java

```
package com.geugeut.guided.guided1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+" dan y = "+y);
    }
}
```

b. B.java

```
package com.geugeut.guided.guided1;

public class B extends A {
    int z;
    int hasil;

    void jumlahXY(){
        hasil = x+y+z;

        System.out.println("jumlah : "+hasil);
    }
}
```

c. Main.java

```
package com.geugeut.guided.guided1;
```

```
public class Main {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();

        System.out.println("Superclass :");
        superclass.x = 3;
        superclass.y = 4;
        superclass.tampilXY();

        //member superclass bisa diakses dari subclassnya
        System.out.println("subclass :");
        subclass.x = 1;
        subclass.y = 2;
        subclass.tampilXY();
    }
}
```

Output :

```
Superclass :
Nilai x = 3 dan y = 4
subclass :
Nilai x = 1 dan y = 2
```

2. Keyword Super

a. Employee.java

```
package com.geugeut.guided.guided2;

public class Employee {
    private String name;
    String departemen;
```

```
    public Employee (String name) {
        this.name = name;
    }

    public void tampilNama() {
        System.out.println("Nama : "+name);
    }
}
```

b. Manager.java

```
package com.geugeut.guided.guided2;

public class Manager extends Employee {
    private String alamat;

    public Manager(String name, String alamat){
        //cara membuat constructor dalam subclass, harus memanggil
constructor superclass dengan super()
        super(name);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("Alamat : "+alamat);
        System.out.println("Departemen : "+super.departemen);
    }
}
```

c. Demo.java

```
package com.geugeut.guided.guided2;
```

```
public class Demo {
    public static void main(String[]args){
        Manager adi = new Manager("Adi", "Jl. Merdeka");
        adi.departemen = "IT";
        adi.tampilInformasi();
    }
}
```

Output:

```
Nama : Adi
Alamat : Jl. Merdeka
Departemen : IT
```

3. **Polymorphism (Overloading Method)**

   a. **Lagu.java**

```
package com.geugeut.guided.guided3;

public class Lagu {
    String pencipta;
    String judul;
    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1,String param2) {
        judul = param1;
        pencipta = param2;
    }
```

```
    void CetakKeLayar() {
        System.out.println("Judul : " + judul);
        System.out.println("pencipta : " + pencipta);
    }
}
```

b. Main.java

```
package com.geugeut.guided.guided3;

public class Main {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("kepastian yang kutunggu","GiGi");

        d.CetakKeLayar();
        e.CetakKeLayar();
    }
}
```

Output:

```
Judul : Lagu 1
pencipta : Tidak dikenal
Judul : kepastian yang kutunggu
pencipta : GiGi
```

4. Polymorphism (Overriding Method)

a. A.java

```
package com.geugeut.guided.guided4;

public class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

b. B.java

```
package com.geugeut.guided.guided4;

public class B extends A {
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar();
        // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

c. Main.java

```
package com.geugeut.guided.guided4;

public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java untuk membandingkan estimasi biaya komputasi dari tiga jenis model machine learning, yaitu model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning. Buat kelas induk yang memiliki atribut nama model dan jumlah data latih, constructor untuk menginisialisasi kedua atribut tersebut, method untuk menampilkan informasi model, serta dua versi method penghitung biaya komputasi method overloading, yaitu tanpa parameter dengan rumus jumlah data latih × 2 dan dengan satu parameter jumlah epoch dengan rumus jumlah data latih × epoch. Buat kelas turunan Regresi yang mewarisi kelas induk dan memiliki atribut jumlah fitur, menggunakan constructor kelas induk, melakukan *method overriding* untuk menampilkan informasi dan menghitung biaya tanpa parameter dengan rumus biaya kelas induk + (jumlah fitur × 10), serta menambahkan versi method penghitung biaya dengan dua parameter jumlah epoch dan ukuran batch menggunakan rumus jumlah data latih × epoch ÷ ukuran batch. Buat kelas turunan Klasifikasi yang mewarisi kelas induk dan memiliki atribut jumlah kelas, menggunakan constructor kelas induk, melakukan *method overriding* untuk menampilkan informasi dan menghitung biaya tanpa parameter dengan rumus biaya kelas induk × jumlah kelas, sedangkan versi penghitung biaya dengan parameter epoch tetap menggunakan implementasi kelas induk. Selanjutnya, buat kelas turunan KlasifikasiDeepLearning yang mewarisi kelas Klasifikasi dan memiliki atribut jumlah layer, menggunakan constructor kelas induknya, melakukan *method overriding* untuk menampilkan informasi dan menghitung biaya tanpa parameter dengan rumus biaya kelas Klasifikasi + (jumlah layer × 200). Pada class Main, buat tiga objek model, yaitu Regresi Linier dengan 500 data latih dan 4 fitur, Decision Tree dengan 800 data latih dan 3 kelas, serta Neural Network dengan 1000 data latih, 5 kelas, dan 3 layer. Simpan ketiga objek tersebut dalam satu array bertipe kelas induk, kemudian gunakan perulangan untuk

menampilkan informasi dan biaya komputasi setiap model serta menghitung total biaya komputasi. Setelah perulangan, tampilkan biaya komputasi 10 epoch untuk model Neural Network dan Regresi Linier, serta biaya komputasi 10 epoch dengan ukuran batch 50 untuk model Regresi Linier. Gunakan access modifier private untuk seluruh atribut serta terapkan konsep inheritance, constructor chaining, method overloading, method overriding, dan polimorfisme sesuai kebutuhan program.

Jawaban :

Model.java

```
package com.geugeut.unguided;

public class Model {
    private String nama;
    private int jumlahDataLatih;

    public Model(String nama, int jumlahDataLatih) {
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNama() {
        return this.nama;
    }

    public int getJumlahDataLatih() {
        return this.jumlahDataLatih;
    }

    public void tampilkanInformasi() {
        System.out.println(
            "Model: " + this.nama
```

```
            + " | Data latih: " + this.jumlahDataLatih
        );
    }

    // Overloading tanpa parameter
    public int hitungBiayaKomputasi() {
        return this.jumlahDataLatih * 2;
    }

    // Overloading dengan parameter epoch
    public int hitungBiayaKomputasi(int epoch) {
        return this.jumlahDataLatih * epoch;
    }
}
```

Regresi.java

```
package com.geugeut.unguided;

public class Regresi extends Model {
    private int jumlahFitur;

    public Regresi(String nama, int jumlahDataLatih, int jumlahFitur) {
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah fitur: " + this.jumlahFitur);
    }
```

```
    public int hitungBiayaKomputasi() {
        return super.hitungBiayaKomputasi() + (this.jumlahFitur * 10);
    }

    public int hitungBiayaKomputasi(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

Klasifikasi.java

```
package com.geugeut.unguided;

public class Klasifikasi extends Model {
    private int jumlahKelas;

    public Klasifikasi(String nama, int jumlahDataLatih, int jumlahKelas) {
        super(nama, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah kelas: " + this.jumlahKelas);
    }

    public int hitungBiayaKomputasi() {
        return super.hitungBiayaKomputasi() * this.jumlahKelas;
    }
}
```

KlasifikasiDeepLearning.java

```
package com.geugeut.unguided;
```

```
public class KlasifikasiDeepLearning extends Klasifikasi {
    private int jumlahLayer;

    public KlasifikasiDeepLearning(String nama, int jumlahDataLatih,
                                   int jumlahKelas, int jumlahLayer) {
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah layer: " + this.jumlahLayer);
    }

    public int hitungBiayaKomputasi() {
        return super.hitungBiayaKomputasi() + (this.jumlahLayer * 200);
    }
}
```

Main.java

```
package com.geugeut.unguided;

public class Main {
    public static void main(String[] args) {
        Regresi regresi = new Regresi(
            "Regresi Linier", 500, 4
        );

        Klasifikasi klasifikasi = new Klasifikasi(
            "Decision Tree", 800, 3
        );
```

```
KlasifikasiDeepLearning neuralNetwork =
    new KlasifikasiDeepLearning(
        "Neural Network", 1000, 5, 3
    );

Model[] daftarModel = {
    regresi,
    klasifikasi,
    neuralNetwork
};

int totalBiaya = 0;

for (Model model : daftarModel) {
    model.tampilkanInformasi();

    int biaya = model.hitungBiayaKomputasi();
    System.out.println("Biaya komputasi: " + biaya);
    System.out.println("--------------------");

    totalBiaya += biaya;
}

System.out.println("Total biaya: " + totalBiaya);

Model modelNeuralNetwork = daftarModel[2];
System.out.println(
    "Biaya 10 epoch (Neural Network): "
    + modelNeuralNetwork.hitungBiayaKomputasi(10)
);
```

```
        Regresi modelRegresi = (Regresi) daftarModel[0];

        System.out.println(
            "Biaya 10 epoch (Regresi Linier): "
            + modelRegresi.hitungBiayaKomputasi(10)
        );

        System.out.println(
            "Biaya 10 epoch batch 50 (Regresi Linier): "
            + modelRegresi.hitungBiayaKomputasi(10, 50)
        );
    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
--------------------
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
--------------------
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
--------------------
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

Program Sistem Perhitungan Biaya Komputasi Model Machine Learning digunakan untuk menampilkan informasi model dan menghitung biaya komputasi

berdasarkan jenis model, jumlah data latih, jumlah fitur, jumlah kelas, jumlah layer, epoch, dan ukuran batch. Program terdiri atas lima class, yaitu Model, Regresi, Klasifikasi, KlasifikasiDeepLearning, dan Main, untuk mengimplementasikan konsep inheritance, method overriding, method overloading, dan polymorphism.

Secara umum, alur program adalah sebagai berikut:

1. Membuat class `Model` sebagai superclass. Class ini memiliki atribut `nama` dan `jumlahDataLatih` bertipe `String` dan `int`. Method `tampilkanInformasi()` menampilkan nama model dan jumlah data latih. Method `hitungBiayaKomputasi()` menghitung biaya dasar dengan mengalikan jumlah data latih dengan 2, sedangkan method dengan parameter `epoch` menghitung biaya berdasarkan jumlah data latih dikalikan epoch. Kedua method `hitungBiayaKomputasi()` menerapkan konsep overloading karena memiliki nama yang sama dengan parameter berbeda.
2. Membuat class `Regresi` sebagai subclass dari `Model`. Class ini memiliki atribut tambahan `jumlahFitur`. Constructor menggunakan `super()` untuk memanggil constructor superclass. Method `tampilkanInformasi()` menampilkan informasi model beserta jumlah fitur, sedangkan method `hitungBiayaKomputasi()` tanpa parameter menghitung biaya dasar ditambah jumlah fitur dikalikan 10. Kedua method tersebut menerapkan overriding karena menyediakan implementasi sendiri untuk method yang diwarisi. Class ini juga memiliki method `hitungBiayaKomputasi()` dengan parameter `epoch` dan `ukuranBatch` untuk menghitung biaya berdasarkan jumlah data latih dikalikan epoch, kemudian dibagi ukuran batch.
3. Membuat class `Klasifikasi` sebagai subclass dari `Model`. Class ini memiliki atribut tambahan `jumlahKelas`. Method `tampilkanInformasi()` menampilkan informasi model beserta jumlah kelas, sedangkan method `hitungBiayaKomputasi()` tanpa parameter menghitung biaya dasar dikalikan jumlah kelas. Kedua method tersebut menerapkan overriding karena memiliki nama dan parameter yang sama dengan method pada superclass, tetapi implementasinya disesuaikan dengan model klasifikasi.

4. Membuat class `KlasifikasiDeepLearning` sebagai subclass dari `Klasifikasi`. Class ini memiliki atribut tambahan `jumlahLayer` dan menggunakan `super()` untuk memanggil constructor superclass. Method `tampilkanInformasi()` menampilkan informasi model, jumlah kelas, dan jumlah layer. Method `hitungBiayaKomputasi()` menghitung biaya komputasi dari class `Klasifikasi` ditambah jumlah layer dikalikan 200. Pewarisan dari `Klasifikasi` yang sebelumnya mewarisi `Model` menerapkan multilevel inheritance.
5. Menginisialisasi dan menjalankan program pada class `Main`. Pada method `main()`, dibuat tiga objek, yaitu Regresi Linier dengan 500 data latih dan 4 fitur, Decision Tree dengan 800 data latih dan 3 kelas, serta Neural Network dengan 1.000 data latih, 5 kelas, dan 3 layer. Ketiga objek disimpan dalam array bertipe `Model`. Program kemudian menampilkan informasi dan menghitung biaya komputasi setiap model menggunakan perulangan, serta menjumlahkan seluruh biaya komputasi sebesar 16.440. Selanjutnya, program menghitung biaya komputasi untuk 10 epoch pada Neural Network sebesar 10.000, untuk Regresi Linier sebesar 5.000, serta untuk Regresi Linier dengan 10 epoch dan ukuran batch 50 sebesar 100. Pemanggilan method melalui array bertipe `Model` menerapkan polymorphism karena implementasi method yang dijalankan menyesuaikan jenis objek sebenarnya.

Program menerapkan enkapsulasi dengan menggunakan akses `private` pada atribut serta menyediakan getter untuk mengakses data yang diperlukan. Constructor menggunakan kata kunci `this` untuk membedakan atribut objek dengan parameter yang memiliki nama sama. Kata kunci `super` digunakan untuk memanggil constructor maupun method dari superclass. Selain itu, program menggunakan array standar bertipe tetap dan casting untuk mengakses method khusus class `Regresi`.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep inheritance memungkinkan suatu class mewarisi atribut dan method dari superclass sehingga dapat digunakan kembali oleh subclass. Keyword super digunakan untuk mengakses constructor maupun method milik superclass. Selain itu, konsep polymorphism

diterapkan melalui method overloading, yaitu penggunaan nama method yang sama dengan parameter berbeda, serta method overriding, yaitu pendefinisian ulang method superclass pada subclass. Penerapan konsep-konsep tersebut pada program perhitungan biaya komputasi model machine learning memungkinkan setiap jenis model memiliki perhitungan dan informasi yang sesuai dengan karakteristiknya. Dengan demikian, inheritance, keyword super, dan polymorphism membantu membangun program yang lebih terstruktur serta mendukung penggunaan kembali kode.